

Module	Fundamentals of Machine Learning: EEEM066
Assessment type	Coursework assignment
Weighting	20%
Deadline	4:00PM, Tue, 2 nd December 2025 (Week 11)
STUDENT URN	6915210
STUDENT Name	Saizalpreet Kaur

Objectives:

- Learn and become familiar with the process of sequential hyperparameter tuning of an entire deep learning pipeline.
- Develop intuition via experimentation on how to use such model design and parameter tuning to improve the performance of a pipeline.
- Interpret experimental results of deep learning experiments and their implications.

Outcomes:

- Navigate general deep learning codebases and modify according to requirements.
- Empirically suggest reasonable values for hyperparameters and select the ones enabling the best performance.
- Understand the implications of changing the backbone architecture of a pipeline.
- Reason why data augmentation works.
- Understand the implications and importance of selecting the best learning rate and batch size.
- Understand how to report experimental results professionally. This includes using tables, plots, and diagrams to communicate ideas and results.

Knife classification in real-world images

For this coursework assignment, you are encouraged to apply what you have learnt from the module materials and your participation in collaborative learning activities like class discussions.

Guidelines

- Submit your assignment as a **Word or PDF document**, along with the supporting evidence as required, via the SurreyLearn submission page before the deadline. Do not submit any other unrelated documents that could flood your submission and mislead the marking process. Only the latest version submitted before the deadline will be assessed if multiple versions exist.
- Label your submission file as: EEEM066-[insert your URN number]-Assignment e.g., EEEM066-1234567-Assignment
- Include a reference list at the end of your assignment, and use the [Harvard](#) or [IEEE](#) referencing style when citing sources. Please use the selected option consistently.
- **Word limit:** Each section of the report should not exceed **200 words** (excluding tables, plots, and graphs). Penalties of up to 50% may be applied for unnecessarily long reports.
- This assignment requires time and effort. It is recommended that you start working on it early to minimise the possibility of experiencing stress around it and avoid missing the deadline.
- You will receive both an overall mark and written feedback on your assignment.
- Extensions for this assignment will only be granted under exceptional circumstances. Extension requests must be submitted via the Extenuating Circumstances (ECs) process, as tutors are not permitted to approve ad hoc extension requests.

Note:

Please follow Surrey's guidelines for [avoiding plagiarism](#).

You must complete this coursework individually. Copying code or text from the internet or another student and including it in your assignment without proper attribution constitutes plagiarism.

Undetected plagiarism undermines the quality of your degree by hindering the assessor's ability to evaluate your work fairly and prevents you from learning through the coursework. If plagiarism is suspected, you will be referred to an Academic Misconduct Panel, which may result in academic sanctions.

All assignments will be checked for plagiarism using Turnitin. Therefore, it is important that you correctly reference sources that have informed your work where appropriate.

Task

Review the instructions for the assignment carefully before starting. This assignment is marked out of 100 and will contribute **20%** to your total module grade. The three sections of the assignment are worth **90 marks**, with an additional **10 marks** allocated to the presentation and clarity of the report.

Background

In recent years, the number of crimes involving sharp objects, such as knives, has increased all across the world. In the year ending March 2022, England and Wales saw around 45,000 offences involving a knife or sharp instrument. This was 9% higher than in 2020/21 and 34% higher than in 2010/11.

Therefore, there is a need to develop an AI-based weapon analysis system that enables officers to identify the make, model, and seller of knives using photographs. Although functional, current knife recognition and labelling methods are beset by inefficiencies due to strained public resources, the possibility of human error, and the UK's expanding digital footprint.

The following figure provides a visual representation of a knife classification system.

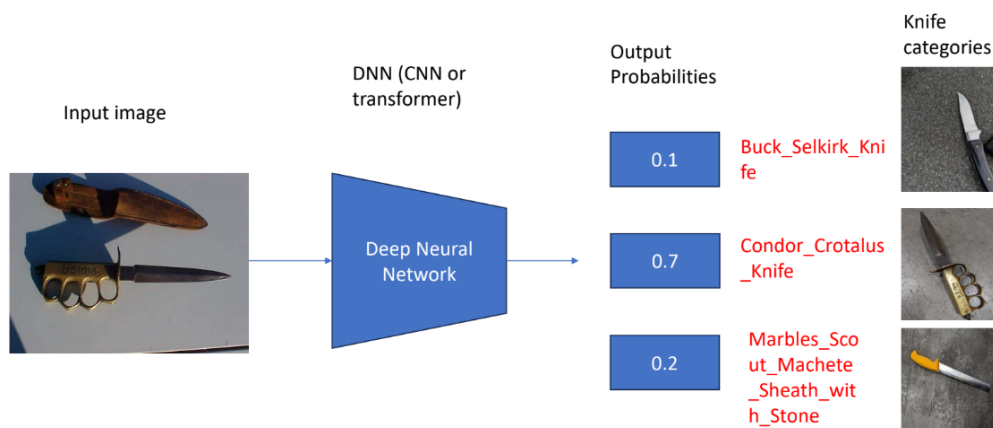


Figure 1: Overview of a knife classification system.

Instructions

In this assignment, you will develop a deep-learning-based system to classify knife images into different categories i.e., classes. You will gain real-world experience in designing, training, and evaluating a deep neural network for accurate, fine-grained knife classification.

You are required to document your findings in a report, divided into three sections as outlined later. Complete each section (in this document) according to the provided guidance. ***The results should be for the test set*** (not for the train/validation set).

Dataset

Use the “Knife classification” training dataset (provided [here](#)), which includes approximately 25,250 knife images categorised into 543 classes, along with a validation dataset of 440 images and a test dataset consisting of 469 images to complete the assignment. The performance of your model will be evaluated on test dataset using the mean Average Precision (mAP) metric.

Important note:

Please do not distribute this dataset or use it for any purposes other than this assignment.

Compute platform

To complete this coursework assignment, you will need to use Google Colab. You are welcome to revisit the lab sheets you engaged with during this module for further guidance that can support you in completing this assignment.

Getting started

To prepare for this assignment, it is recommended that you engage with the following readings, available via the Reading List, before starting work on developing your knife classification system:

- “Fine-grained image analysis with deep learning: A survey” by Wei Xiu-Shen, Jianxin Wu, and Quan Cui
- “Weapon classification using deep convolutional neural network” by Neelam Dwivedi, Dushyant Kumar Singh, and Dharmender Singh Kushwaha

Additionally, you are encouraged to revisit the lectures in this module and the lab sessions on using Python and PyTorch to train and test CNNs and Vision transformers for image classification. The PyTorch Introductory lab sheet and the lecture slides on these topics are available on SurreyLearn.

Support in programming

You can consult this [GitHub repository](#) for a reference code base. Should you experience any problems during the development of your knife classification, please feel free to open an issue.

Deliverables

For this assignment, you are required to run your project in the Colab. For the report, you must document the experiments performed using your software.

Additional instructions

Keep the following guidance in mind as you work on producing the deliverables for this assignment:

Hyperparameters

For the parameters that are not learnable, you can set them before starting the training.

Format

- Ensure your responses to each question are on the spot as shown in Figure 2. Focus on the specific question and not move away.

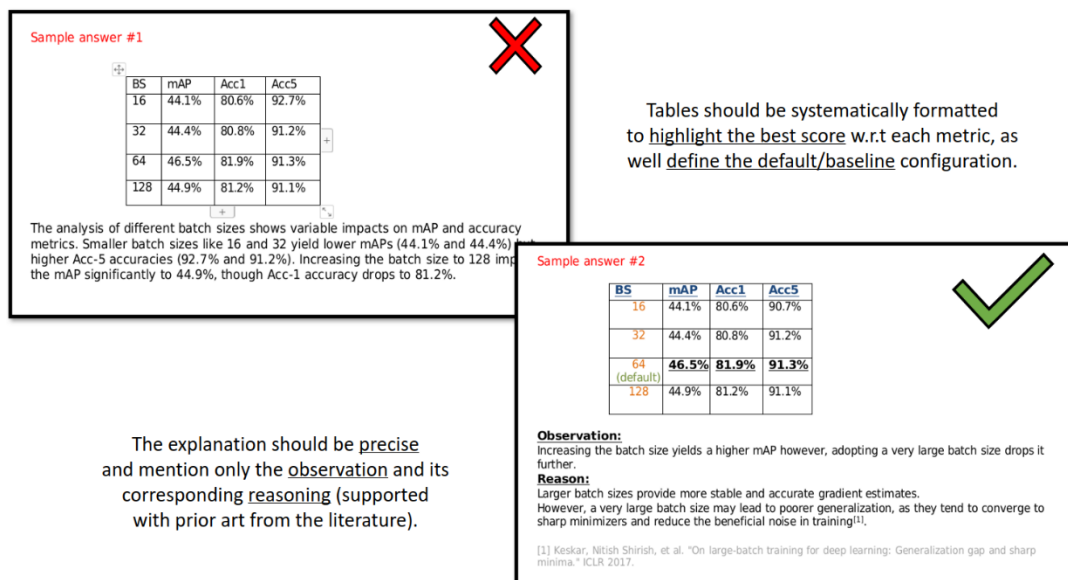


Figure 2: Response example.

Log files

- Submit your own log files from your codebase. You are required to submit a separate log file for each experiment-based question.
- Do not change the log file structure. Each log file is watermarked and unique to each run.
- Name each log file according to the section number and question number: (log_sec{Section_num}_q{Question_num}.txt). For example, the log file generated for an experiment that corresponds to Section 1 Question 2, should be named as "log_sec1_q2.txt".

Presentation and clarity

In addition to the technical content, the presentation and clarity of your writing will be assessed.

Model performance metrics

While different metrics are available, it is recommended that you prioritise mAP when selecting the best-performance models.

Start your assignment

Write your assignment in the designated space provided below. Each of the sections outlines the tasks required and specifies the information you must include in your report for that section.

Section 1: Familiarity with the code provided (40 marks)

1. Run the code using the default settings. Discuss the steps involved in the training and evaluation process and explain the impact of the observed performance using an appropriate metric. (20 marks)
2. Apply a different CNN variant from the one used in the code provided. Critically compare the results with those observed in Question 1. (10 marks)
3. Apply one more neural network architecture (CNN or Vision transformer) different from those in Questions 1 and 2. Critically discuss and compare the results from these previous two steps. (10 marks)

Note:

For Questions 2 and 3, ensure the new architecture is explicitly specified in your shell script command if you are using one from the codebase.

Start writing here:

1.1:

A basic Deep learning pipeline is followed for this project. A train:val:test split is used. A dataloader loads the dataset and resizes the images, normalizing and applying augmentations (colour jitter as default). Pre-trained models from timm are used. Each training epoch loads mini batches, passes through model to compute cross entropy loss and update weights using Adam optimiser A learning rate scheduler is also used to stabilize learning over time. Saved checkpoint is loaded for evaluation and mAP is computed.

The primary metric is mean Average Precision. It handles class imbalance and evaluates the precision-recall trade-off. It captures the model's performance across all classes. A higher mAP score indicates a better performance.

Sr. No.	Architecture used	Test mAP score
1	Tf_efficientnt_b0 (default)	0.449
2	Resnet50	0.477
3	Vit_bse_patch16_224	0.462

Table1: mAP scores for the test set for different architectures

Observation: Efficientnet gives the worst result.

Reason: This is because of the compactness of the model which lacks the depth to capture fine-grained intra-class variance [1].

1.2:

Observation: ResNet50 performs the best out of the three models, giving good score-time trade-off.

Reason: Resnet's deep residual architecture with skip connections helps it learn subtle visual differences, making it suitable for fine-grained image tasks [2].

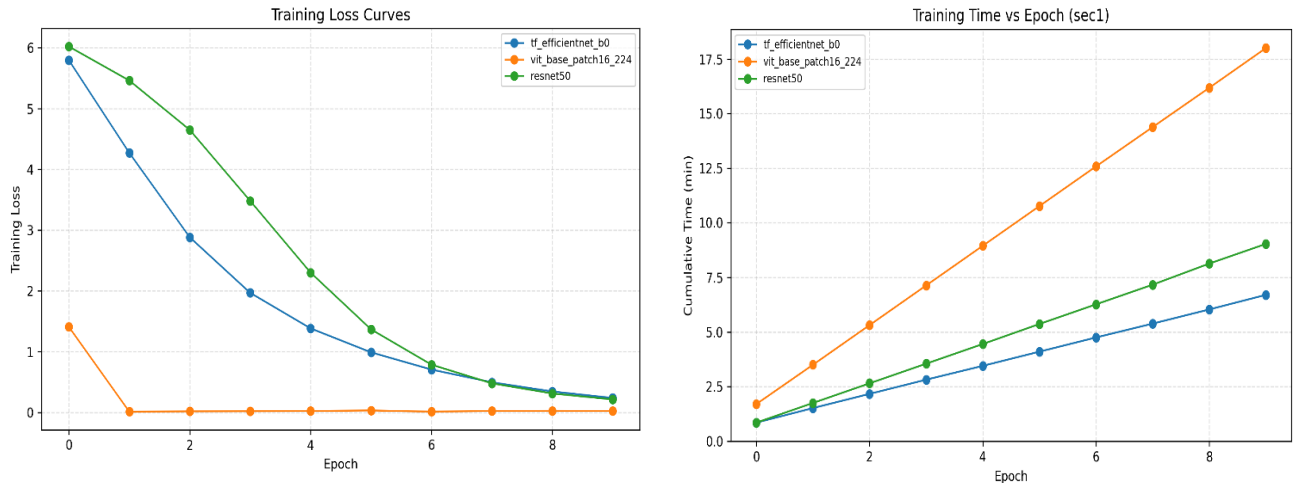


Fig 1: (a) Training loss curve for the three models. (b) Time taken to train each model, ViT takes the longest time but fails to give a comparable performance.

1.3:

Observation: ViT performs better than efficientnet.

Reason: Big models (ViT) require larger training data to perform well and generalize [3]. The loss curve suggests an overfitting issue potentially caused by the smaller dataset.

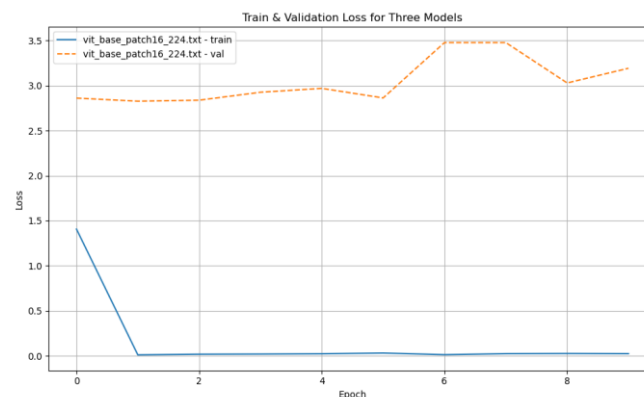


Fig 2: Training and Validation losses for ViT

Section 2: Dataset preparation and augmentation experiment (30 marks)

Begin with the default data augmentation setting of the default command line, which uses colour jittering.

1. Further append on top two additional data augmentation techniques, one at a time e.g., Default + “random rotation”, Default + “horizontal flip”. Compare the results with the default configuration in the provided code and discuss the differences in performance. (20 marks)
2. Combine the augmentation techniques from Question 1 to find the best-performing combination. Highlight any improvement or drop in the overall score. (10 marks)

Start writing here:

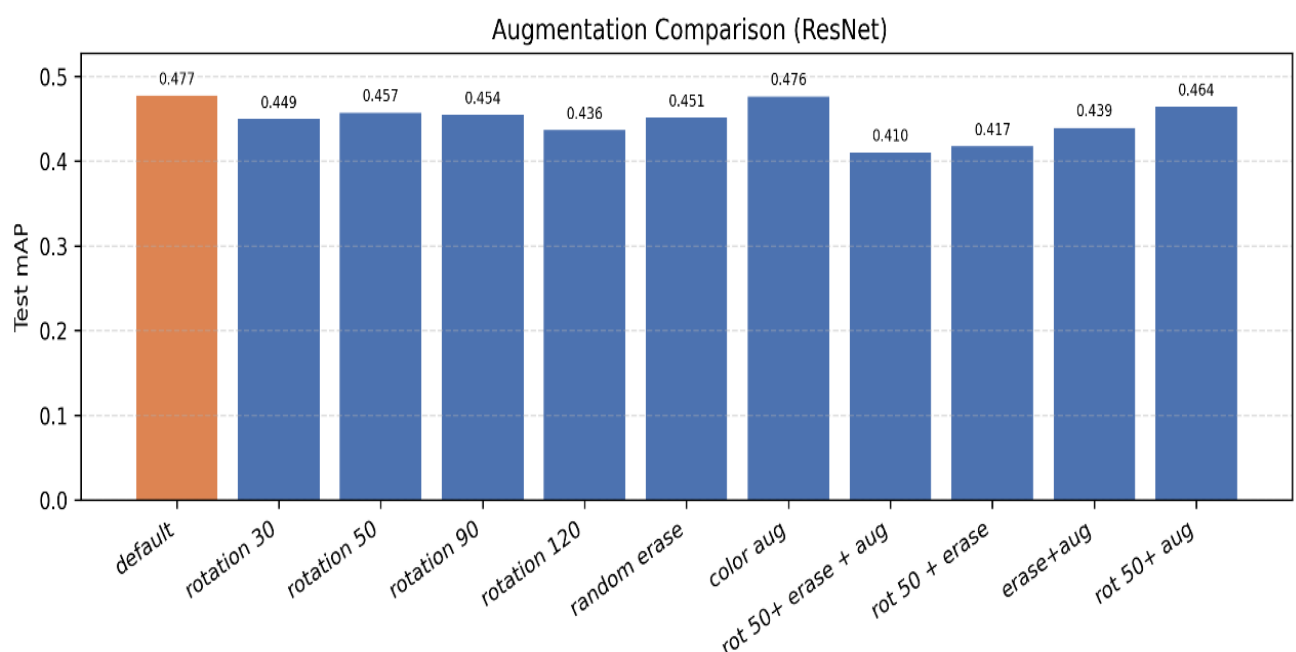


Fig 3: The test mAP for different Augmentations and their combinations

NOTE: Resnet50 was chosen for section 2 and 3

3.1:

Observation: Small Rotations had minimal effect but larger angles reduced scores significantly, random erasing degraded performance while colour augmentation almost maintained it.

Reason: For rotation, local features are easily disrupted by large transformations for fine-grained tasks [4]. Random Erase likely occluded some critical discriminative parts. Such aggressive occlusions can hinder the process where local details matter the most [5]. Colour Augmentation refrains from physically distorting the data or erasing details, It maintains local textures and knife details because it is not as harsh as the other techniques and improves robustness of the model, while retaining performance.

Sr. NO.	Rotation	Random Erase	Color Augmentation	Test mAP
1 (default)	0	False	False	0.477
2	30	False	False	0.449
3	50	False	False	0.457
4	90	False	False	0.454
5	120	False	False	0.436
6a	0	True	False	0.451
7	0	False	True	0.476
8	50	True	False	0.417
9	50	False	True	0.464
10	0	True	True	0.439
11	50	True	True	0.410

Table 2: Test mAP values for all the data augmentations used

3.2:

Observation: Each combination gave worse results than the individual augmentations.

Reason: While the augmentations individually help in regularization, when applied together, they disrupt the key discriminative parts of the images which are required for such fine-grained classification tasks. Random Erasing combined with anything gave bad results because it usually tends to remove the parts which might be of importance for training, say the curves of the blade. Further augmentation with it just leads to underfitting (smaller dataset). Again, colour augmentation with small rotation gave comparably better results because of the same reason that it retains important local features.

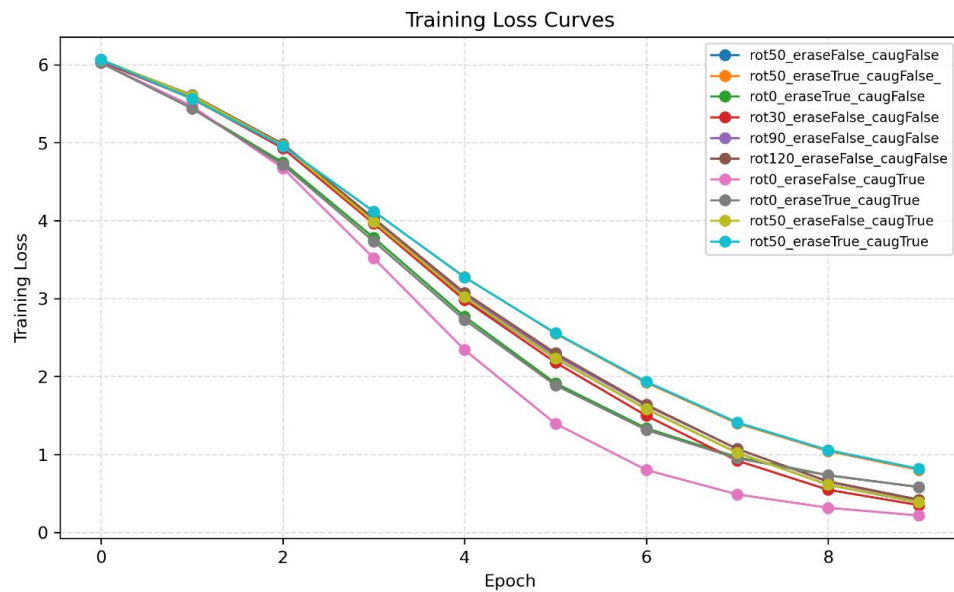


Fig 4: Training loss curves for each experiment

Section 3: Exploration of Hyperparameters

(20 marks)

Start with the default learning rate (LR) and batch size (BS).

1. Exploration of LR. (10 marks)

- Experiment with three values of LR (in addition to the default value).
- Discuss the observed impact of each value on overall performance.

2. Exploration BS. (10 marks)

- Using the best LR value from the experiments in Question 1, test three different values of the BS in addition to the default value.
- Discuss how the changes in BS affect overall performance.

Start writing here: 3.1:

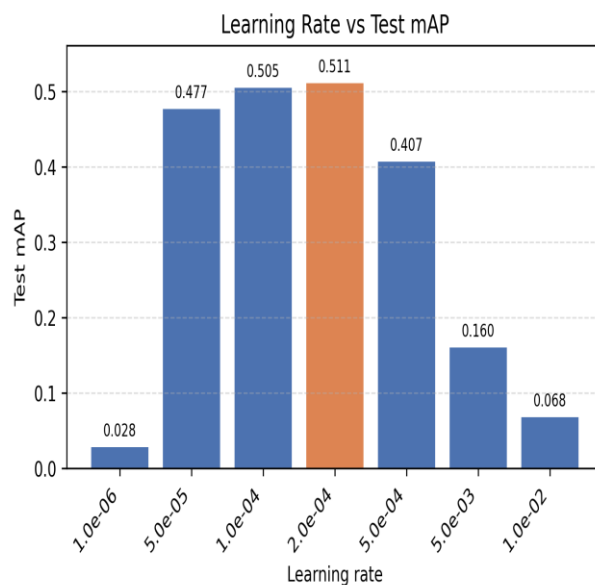


Fig 5: mAP values for varied Learning rates

Sr.No.	Learning Rate	mAP
1	0.0005	0.407
2	0.005	0.160
3	0.000001	0.028
4	0.0001	0.505
5	0.01	0.068
6	0.00005 (default)	0.477
7	0.0002	0.511

Table 3: mAP values for different values of learning rates

Observation: The best test mAP was achieved at 0.0002, while both higher and lower rates performed poorly.

Reason: For extremely small learning rates, the model's learning process slowed down significantly as can be seen in the loss curves. The step size seemed to be too small to escape local minima or to converge within the number of epochs. Conversely at higher values, the optimization process seems to overshoot minima, which prevents an effective convergence. [6]. Moderate learning rate values allowed stable convergence as seen in the table above.

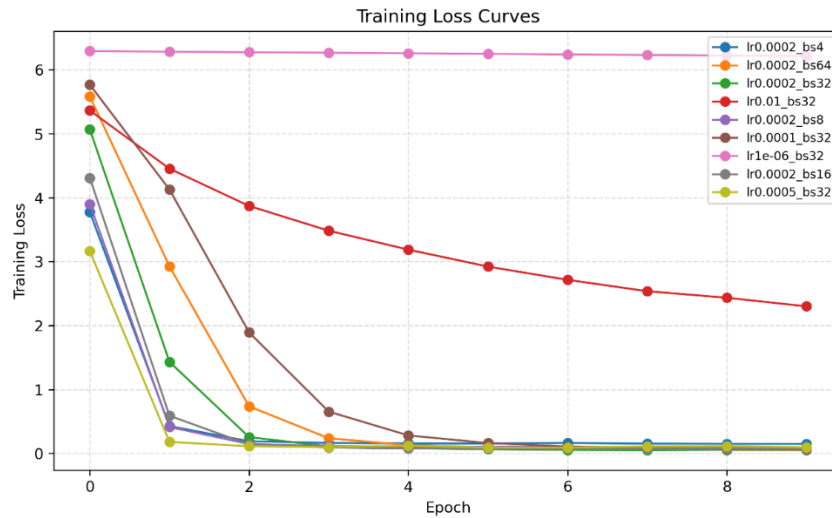


Fig 6: Training loss curves for different learning rates and batch sizes

3.2:

Observation: The default batch size is the best batch size and gives better results.

Reason: Small batches result in noisy gradient estimates. This can increase generalization in some settings but often slows down convergence and destabilizes the learning process, especially for fine-grained classification. While larger batches provide more stable gradient estimates, very large batch sizes can lead to poorer generalization, especially for smaller datasets [7]. They need a higher learning rate or longer training for better convergence. Batch size 32 offered the best balance, providing stable learning without suppressing important variations in gradient direction.

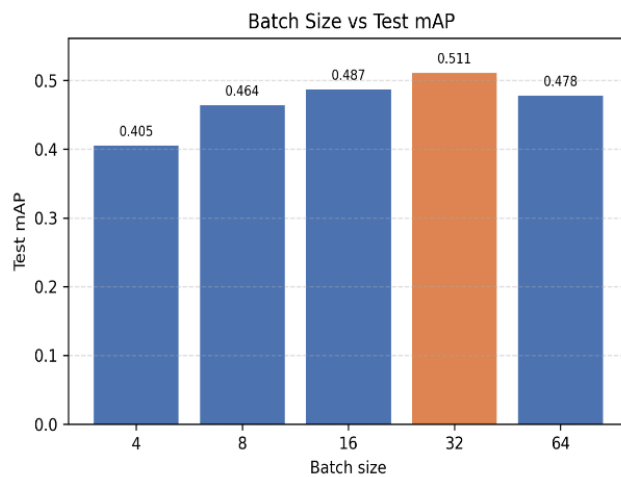


Fig 7: mAP scores for various batch sizes

Sr. NO.	Batch Size	Test mAP
1	4	0.405
2	8	0.464
3	16	0.487
4	32(default)	0.511
5	64	0.478

Table 4: mAP values for various batch sizes

Bibliography

- [1] M. a. Q. L. Tan, "Efficientnet: Rethinking model scaling for convolutional neural networks," *International conference on machine learning.*, PMLR, 2019..
- [2] L. e. al., "Comparison of Four CNN-Based Algorithms for Sports Image Classification," *Proceedings of CISIS* , pp. 179-181, 2023.
- [3] P. & S. A. Jeevan, "Which backbone to use: a resource-efficient domain specific comparison for computer vision.," no. arXiv preprint arXiv:2406.05612., (2024).
- [4] X.-S. e. a. Wei, ""Fine-grained image analysis with deep learning: A survey," *IEEE transactions on pattern analysis and machine intelligence* 44.12 , pp. 8927-8948, 2021.
- [5] Z. e. a. Zhong, "Random Erasing Data Augmentation," *AAAI*, 2020.
- [6] Y. Bengio, " "Practical recommendations for gradient-based training of deep architecture".," *Neural networks: Tricks of the trade: Second edition. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012. 437-478.*
- [7] S. L. e. a. Smith, "Don't Decay the Learning Rate, Increase the Batch Size," *ICLR*, 2018.
- [8] J. Maurício, I. Domingues and J. Bernardino, "Comparing Vision Transformers and Convolutional Neural Networks for Image Classification: A Literature Review," *Applied Sciences. 2023; 13(9):5521.*

Rubric

Section 1: Familiarity with the code provided (40 marks)

Task 1.1: Running and understanding the code (20 marks)

- **Understanding the code execution**
 - *Criteria:* Clear understanding and explanation of the code flow and processes.
 - *Indicators:* Detailed description of data loading, model initialisation, training loop, and evaluation steps.

(5 marks)
- **Metric explanation and performance analysis**
 - *Criteria:* Appropriate choice and understanding of the evaluation metric(s) (e.g., accuracy, mAP).
 - *Indicators:* Correct calculation and interpretation of the metric values. Discussion on what these values imply about the model's performance.

(10 marks)
- **Discussion of observed performance**
 - *Criteria:* Insightful discussion on the observed results.
 - *Indicators:* Analysis of performance patterns, potential overfitting/underfitting issues, and suggestions for improvement.

(5 marks)

Task 1.2: Applying a different CNN variant (10 marks)

- **Implementation of a new CNN variant**
 - *Criteria:* Correctly modifying the code to implement a different CNN architecture.
 - *Indicators:* Model architecture is different from the provided one, and code runs without errors.

(4 marks)
- **Comparison and critical analysis**
 - *Criteria:* Critical comparison between the original and new CNN results.
 - *Indicators:* Use of quantitative results to support comparison, discussion on potential reasons for performance differences.

(6 marks)

Task 1.3: Applying another neural network architecture (10 marks)

- **Implementation of a new neural network**
 - *Criteria*: Correctly implementing a neural network architecture different from those in Tasks 1.1 and 1.2.
 - *Indicators*: Model architecture is distinct, and code runs correctly.

(4 marks)
- **Comprehensive comparison and analysis**
 - *Criteria*: Deep comparative analysis between all three architectures.
 - *Indicators*: Insightful discussion on the strengths and weaknesses of each model, based on quantitative results and qualitative observations.

(6 marks)

Section 2: Dataset preparation and augmentation experiment (30 marks)

Task 2.1: Augmentation techniques (20 marks)

- **Implementation of additional augmentations**
 - *Criteria*: Successful addition of two new augmentation techniques.
 - *Indicators*: Correct application and explanation of chosen techniques, code runs without errors.

(8 marks)
- **Performance comparison and analysis**
 - *Criteria*: Comparative analysis of the impact of each augmentation setting.
 - *Indicators*: Clear explanation of performance differences between default and new settings, supported by quantitative results.

(8 marks)
- **Discussion on results and insights**
 - *Criteria*: Insightful interpretation of how augmentations affected model performance.
 - *Indicators*: Discussion on potential reasons for performance changes and implications for model robustness.

(4 marks)

Task 2.2: Best-performing augmentation combination (10 marks)

- **Combining techniques and experimentation**

- *Criteria*: Correct implementation of combined augmentation techniques.
- *Indicators*: Code successfully combines the techniques and executes without errors.

(5 marks)

- **Analysis of improvement or decline**

- *Criteria*: Detailed discussion on whether the combined augmentation improved performance.
- *Indicators*: Use of appropriate metrics to highlight any performance change, supported by logical reasoning.

(5 marks)

Section 3: Exploration of hyperparameters (20 marks)

Task 3.1: Learning rate exploration (10 marks)

- **Experimentation with different LR values**

- *Criteria*: Correct experimentation with three additional LR values.
- *Indicators*: Clear documentation of the chosen LR values and implementation.

(4 marks)

- **Performance analysis of each LR**

- *Criteria*: Thorough analysis of the effect of each LR value on performance.
- *Indicators*: Use of metrics to compare performance across LR values, discussion on the reasons behind observed changes.

(6 marks)

Task 3.2: Batch size exploration (10 marks)

- **Experimentation with different BS values**

- *Criteria*: Correct experimentation with three additional BS values.
- *Indicators*: Clear documentation of the chosen BS values and implementation using the best-performing LR from Task 3.1.

(4 marks)

- **Performance analysis of each BS**

- *Criteria*: Detailed analysis of the impact of each BS value on performance.
- *Indicators*: Use of metrics to compare performance across BS values, discussion on the reasons behind observed changes.

(6 marks)

Report writing and presentation (10 marks)

- Figures should be well-presented, with readable axes and labels.
(2 marks)
- The writing should be clear, grammatically correct, and easy to follow.
(6 marks)
- Tables are used when discussing results involving multiple hyperparameters values
 - *Criteria*: Clarity, structure, and depth of the written report.
 - *Indicators*: Logical flow, clear explanation of methods and results, proper use of figures/tables to support the analysis.
(2 marks)

Total: 100 marks